4　形式意味論の基礎

講義の概要

(1)　意味論研究の歴史

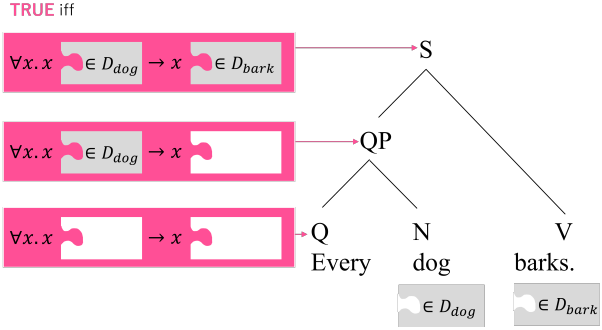
a.　「意味概念」を研究する

b.　「意味機能」を研究する：

(i)　部分的合成性：認知意味論

(ii)　合成性：哲学（論理学）→ 形式意味論

(2)　やりたいこと：ツリーにおける各ノードの意味機能を推定



a.　ツリーの形はどう決まるの？　⇒文の構造

b.　パズルの比喩をやめる　⇒解釈関数・ラムダ式

c.　ピースの形を正確に分類　⇒意味タイプ

統語論の基礎 1

構成素テスト

ある複数の言語表現が構成素をなすかを確認するため検証する手続きのこと。

構成素（Constituent）：句・節といった単語たちが寄せ集まってできるかたまりのこと。

ここがポイント！ 代表的な構成素テスト

(3) 置換テスト（Replacement test）：
ある文字（発音）列が、一単語で置き換えられる
→ それが構成素（例：代名詞 it や代動詞句 do so）

ほかにも様々なテストが考案されているが、詳細は統語論の授業に譲り、文の階層構造はもう分かっているという前提で話を進める。

a. 例 1：(i) My father visited the house.
(ii) He visited the house.

b. 例 2：(i) My father visited the house.
(ii) My ??? the house.

c. 例 3：(i) My father visited the house.
(ii) My father did so.

d. 例 4：(i) My father visited the house.
(ii) ??? the house.

例題 4-1 【内容理解】構成素テスト ★

次の下線部の中から構成素であるものを選び、記号で答えよ。

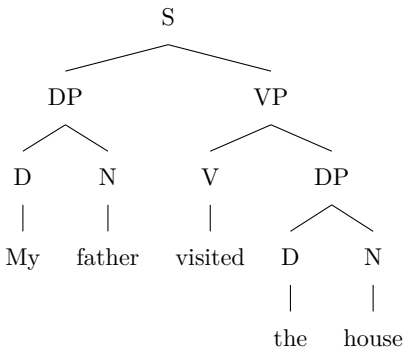
(4) a. (a)The man (b)played the (c)piano with (d)her sister.
b. (e)Every dog barked, so (f)some cats left the room.
c. (g)Every dog barked, so (h)some cats left the room.

統語論の基礎 2

ツリー

文の中の階層関係（構成素の包含関係）を表す手法。

ここがポイント！ ツリー表示



【解説】構成素テストの結果にしたがって、グループになっている単語たちを線でつなぎ、各接点にそれが何のかたまりなのか（例：動詞のかたまりVP）を記入していく。

統語論の枠組みが変わると、同じ文に異なるツリーが提案されることもあるが、少なくとも入門の段階では、与えられたツリーが正しいという前提の下で、各ノードの意味がどうあるべきか推定する能力を伸ばすことに集中しておこう。

ここがポイント！ 記法：品詞のラベル

記号	和名	具体例
S	文	*Alice runs.*
N	名詞	*Alice, dog*
D	決定詞	*the, a*
A	形容詞	*red, tall*
V	動詞	*walks, meets*
Adv	副詞	*quickly, always*

記号	和名	具体例
Aux	助動詞	*have, be*
Q	量化詞	*every, some*
T	時制	*-ed, will*
C	補文標識 (従属接続詞)	*that, if*
Conj	等位接続詞	*and, but*

T は Tense、C は Complementizer の略

例題 4-2 【内容理解】ツリーを描く ★★

次の文の構造をツリーを用いて表現しなさい。

(5) Every dog chases every cat.

形式意味論の基礎 1

解釈関数 ⟦　⟧

言語表現（文／単語など）を入力すると、その意味（指示対象や真理値のこと）を返す関数。次のように表す：

⟦ 言語表現 ⟧ =（解釈された）意味

「山田」という言語形式と人物の山田は別物。

本当はもう少し複雑で、「外延」と「内包」というものの区別を細かくする必要があるのだが、それは「名詞」の意味論を扱う時に掘り下げる。

ここがポイント！ 解釈関数。読んで字のごとく、意味解釈を返す関数。

関数であることを強調するなら、(6) と書いてもよいのだが、せっかくそれ自体が括弧の形をしているので、(7) に示すように、入力値をその中に書く。⟦ **言語表現** ⟧ と書き、「『**言語表現**』の持つ意味」と読む。(7) なら「『**Pochi barks** という文』の持つ意味」ということ。

(6) ⟦　⟧(**Pochi barks.**)

(7) ⟦ **Pochi barks.** ⟧

同じことは、小学校からおなじみの + などの記号についても言える。これも関数なので、$1+2$ は、関数的な考え方を意識すれば $+(1,2)$ と表記しても同じこと。一部の関数には、そっちの方が色々便利なので、特殊な表記が許されているというわけだ。

ここがポイント！ 条件つきの返り値

(7) を踏まえると前頁の内容は次のように書き表される。

(8) a. ⟦ **Pochi barks.** ⟧ =TRUE, if $p \in D_{bark}$.
　　b. ⟦ **Pochi barks.** ⟧ =FALSE, if $p \notin D_{bark}$.

でも、いつも二つ書くのは面倒なので、(10) のように真理条件を一つにまとめる。

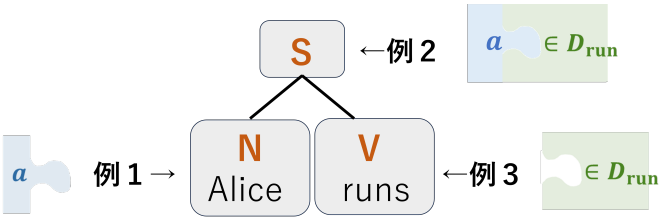
(9) ⟦ **Pochi barks.** ⟧ =TRUE, iff $p \in D_{bark}$.

一般的な形式を取り出しておくと、こんな感じだ。

(10) ⟦ **言語表現** ⟧ = 真理値, iff 真理条件.
　　（入力）　　（出力）

書き方はだいぶ数学チックになってしまったが、捉えたいのは、述語の意味の違いは真理条件で捉えるということ。それさえ押さえられていれば、全然大丈夫！ 前章で見た if and only if (iff) が活躍。

ここがポイント！ 解釈関数 ⟦ 入力 ⟧ = 出力



(11) 例 1 : 固有名詞 ⇒「指示対象」

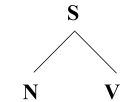
a. ⟦ **Alice** ⟧ = a

b. ⟦ **Bob** ⟧ = b

(12) 例 2 : 文 ⇒「真理値」

a. ⟦ **Alice runs** ⟧ = TRUE, iff $a \in D_{run}$.

b. ⟦ [$_S$ **Alice runs**] ⟧ = TRUE, iff $a \in D_{run}$.

c. ⟦  ⟧ = TRUE, iff $a \in D_{run}$.

(13) 例 3 : 自動詞 ⇒「関数」

a. ⟦ **walk** ⟧ = 指示対象 x を入力値として取って、$x \in D_{walk}$ なら「真」、それ以外の条件では「偽」という出力をする関数

b. ⟦ **run** ⟧ = 指示対象 x を入力値として取って、出力値が、$x \in D_{run}$ なら「真」、それ以外の条件では「偽」という出力をする関数

実は、代名詞 ⟦ **she** ⟧= *alice* とか、別の固有名詞なのに同じ人を指すとき ⟦ 工藤新一 ⟧ = ⟦ 江戸川コナン ⟧（あるいは、指せないとき）とかを考え始めると、なかなか難しい。

これを less wordy な形で表現したいというのが、次のページで習うラムダ式。

例題 4-3 【内容理解】解釈関数 ★

次の各説明に対し、正しければ T、間違っていれば F と記しなさい。

(14) a. 文 He met her. において [$_{VP}$ met her] は構成素である。この構成素に対応する意味は ⟦ [$_{VP}$ **met her**] ⟧ と表現される。

b. ⟦ [$_S$ **Ann met Bob**] ⟧ の返り値は、Ann met Bob である。

形式意味論の基礎 2

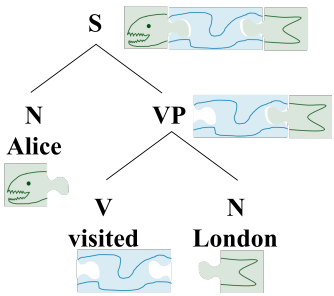
合成性の原則

言語表現全体の意味が、(i) それを構成する単語（または形態素／素性）の意味と (ii) その言語表現の構造から決まる、という原則。

ここがポイント！　合成性の原則

額縁（S）まできたら、勝手にピースを増えてるとか、減ってるとかはご法度。

品詞のラベルの後ろにつく P は、句（phrase）の略で、VP は動詞単体ではなく、動詞句（Verb Phrase）。



合成性の原則のイメージ

【解説】(i) 額縁全体の絵が、パズルのピースから決まるように、Alice runs という文の意味も、Alice という単語と runs という単語から決まる。(ii) ただし、単語にはピースをはめる順序が存在し、文の構造に沿って合成は進む。この文の構造を示しているのがツリー（樹形図）である。この図で言えば、「胴体」はまず「尻尾」とくっつかなければならず、さきに「頭」とくっつけてはいけないということ。

例題 4-4　【内容理解】合成性の原則　★★

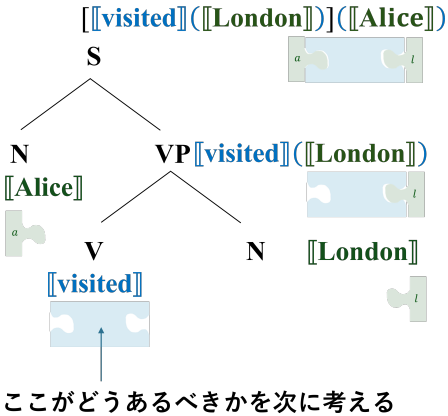
次の各説明は、誤りである。例を添えて、この説明に反論しなさい。

(15)　文の意味は「単語の意味」の単純な足し算で決まるため、単語を並べる順番を入れ替えても文全体の意味が変わることはない。

例題 4-5　【内容理解】合成性の原則　★★

合成性の原則は、「原則」という言葉が示すように、言語の意味がしたがう中心的な原理ではある一方で、絶対的なものではない。例外となる表現を 3 つ考えて示せ。

ここがポイント！ 解釈関数を用いた表現



解釈関数を用いた表現

【解説】V と N から VP の意味が決まる。そこで関数 V が N（目的語）をとって VP を返す、と考える。そこで、まず他動詞 visit(ed) は、二つ穴が（入力値）を求める関数だと考えておく。

(16) ⟦ visited ⟧：

統語論の知識から、他動詞は、まず目的語とひっつくいて構成素を築くので、まず、Longon が引数に取られて visited London という意味の塊ができる。数学で $f(a)$ と書くように、

(17) ⟦ visited ⟧(⟦ London ⟧)：

と書く。また VP 自体がさらに N（主語）を取って、S を出力する。(17) 自体が引数（argument）を取るので、(18)a のように書きたいところだが、毎回四角で囲うのは大変なので、(18)b のように大かっこで囲ったり、あるいは、もうわかるでしょと言うテンションの時は (18)c のように大かっこさえ省略されるときもある。

(18) a. $\boxed{\text{⟦ visited ⟧(⟦ London ⟧)}}$(⟦ Alice ⟧)：
b. [⟦ visited ⟧(⟦ London ⟧)](⟦ Alice ⟧)
c. ⟦ visited ⟧(⟦ London ⟧) (⟦ Alice ⟧)

何かの頭文字で λ というわけではなく、山型の記号（$\hat{x}$）を使おうとしたところ、印刷機が対応しておらず、形が似ていたギリシャ文字の λ（ラムダ）で代用したのが始まりという説もがあるが、本当かどうかは定かではない。

形式意味論の基礎 3

ラムダ演算子

変数が何かを一目でわかるようにするために、式の冒頭に、変数にラムダ（λ）をつけて表現する。

ここがポイント！ ラムダ演算子を学ぶ理由：変数が何か簡潔に表す

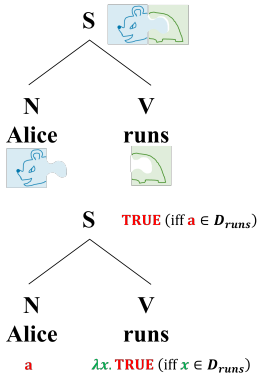
この書き方には、次のような嬉しい性質がある。(1) 入力値と出力値が何か「.」が区切って教えてくれている。(2) 変数（変項）と定数（定項）を区別して表している。(3) 関数が連続適用されるときの項を取る順番を教えてくれている。

Before　$[\![$ **run(s)** $]\!] =$ 指示対象 x を入力値として取って、$x \in D_{run}$ なら「真」、それ以外の条件では「偽」という出力をする関数

After　$[\![$ **run(s)** $]\!] = \lambda x.$ TRUE, iff $x \in D_{run}$.
（入力）.（出力）, iff（条件）

それでも分かりにくいという人にとっておきの「裏技」がある。それは λ ではなく「入力」の「入」という字を書いていると思うというもの。λx を「入力は x です」、「ピリオド以降 iff より手前」を「出力」、残りを「条件」と読めばいい。

【解説】$[\![$ **Alice runs** $]\!]$ =TRUE, iff $a \in D_{run}$ なので、$[\![$ **run(s)** $]\!]$ の意味は、そこから $[\![$ **Alice** $]\!]$ の意味を「引き算した」残り、すなわち定まった a という値が、変数 x に置き換えられたものということになる。変数を指定する方法には、色を付けたり（例：TRUE, iff $x \in D_{run}$）、四角で囲ったり（例：TRUE, iff $\boxed{x} \in D_{run}$）何でも良いのだが、慣習的に λx のように書く書き方が採用されている。



λ は自由変項を教えてくれる記号でもあるというわけだ。

ここがポイント！ 項の分類

	$f(a).$	「a が	f する」	a は定項
$\exists x.$	$f(x).$	「あるモノが	f する」	x は束縛変項
$\forall x.$	$f(x).$	「すべてのモノが	f する」	x は束縛変項
$\lambda x.$	$f(x).$	「x が	f する」	x が自由変項

例題 4-6 【内容理解】ラムダ演算子（正誤問題） ★★

次の記述が正しいか誤りか答えよ。

(19) a. λx は、式の中で変数が何であるかを示す記号である。
b. $\lambda x.f(x)$ において、x は自由変項である。
c. $\forall x.f(x)$ において、x は自由変項である。
d. $\lambda x.f(x)$ は、f が固定で入力値が x である関数を表す。
e. $\lambda f.f(x)$ は、x が固定で入力値が x である関数を表す。

例題 4-7 【内容理解】ラムダ式（選択問題） ★

各問いについて、正しい選択肢を一つ選び、記号で答えなさい。

(20) 次のうち、述語 **run** の意味を表すラムダ式として最も適切なものはどれか。
① $\lambda x.x \in D_{run}$
② $\exists x.x \in D_{run}$
③ $\forall x.x \in D_{run}$
④ $run(a)$

(21) 次のうち、$\lambda x.f(x)$ の説明として正しいものはどれか。
① すべての x が f することを表す
② ある x が f することを表す
③ 入力 x を取り、$f(x)$ を出力する関数を表す
④ x が存在しないことを表す

(22) 次のうち、関数を表す式はどれか。
① $\lambda x.f(x)$
② $\forall x.f(x)$
③ $\exists x.f(x)$
④ $f(a)$

例題 4-8 【内容理解】ラムダ式の書き方 ★★

下記の空欄を埋めよ（集合の表記は自ら定義してよい）。

格助詞「が」は無視してください。

(23) a. ⟦ **プーさんが歩く。** ⟧ = _____
b. ⟦ **プーさん** ⟧ = _____
c. ⟦ **歩く** ⟧ = _____

形式意味論の基礎 4

タイプ

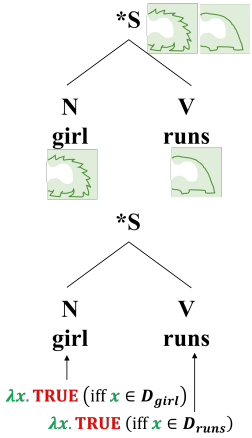
その意味が属する集合のこと。

ここがポイント！ 捉えたいこと

【解説】同じ名詞なのに、普通名詞はダメだが、固有名詞は主語になれる。

(24) a. *Girl runs.
b. Alice runs.

ここがポイント！ 意味にタイプ（種類）を考える



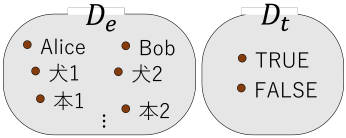
【解説】パズルを完成させるためには、各ピースの凸凹の形状を知ることが大切だ。意味も同じで、組み合わせられるのは特定の種類のものに限られる。これを形式意味論では、「意味のタイプがあっていないから」と表現する。例えば、上記の(24)a がダメなのは、run(s) が「定項（個体）」を入力値として求めていたのに、girl は「関数」だからだ、というような説明をする。この点を正確に捉えるために、集合論の言葉を使って、意味のタイプを表現する。

run には、固有名詞の意味は "はまっても"、普通名詞の意味は "はまらない" ということだ。

ここがポイント！ 二つの基本タイプ

(25) D_e := { 個体の集合 }

(26) D_t = { TRUE, FALSE }



ここがポイント！ 基本タイプの例

【解説】ある言語表現 **X** の意味が、タイプ A であるとき、$[\![\mathbf{X}]\!] \in D_A$ と表す。

固有名詞の意味 固有名詞の場合、次のようになる。

(27) a. $a \in D_e$
b. $[\![\textbf{Alice}]\!] = a \in D_e$
c. $[\![\textbf{Alice}]\!] \quad \in D_e$

文全体の意味 文全体のタイプは、次のようになる。

(28) a. $\text{TRUE} \quad \in D_t$
b. $[\![\textbf{Alice runs}]\!] = \text{TRUE (iff } a \in D_e) \in D_t$
c. $[\![\textbf{Alice runs}]\!] \quad \in D_t$

タイプが A である意味を集めて集合を作ったら、[[**X**]] もその one of them だということ。

読み方は二通り。「**X** の意味はタイプ A です」とコピュラ文のように読むか、または「タイプ A である **X**」と後置修飾で読む。(27)c では、まさに後者がよい。「入力は、『タイプ e である x』です」

ここがポイント！ 関数のタイプ（派生タイプ）の例

述語の意味 述語（関数）のタイプは、入力と出力の集合が分かれば、規定できるので $\langle$入力集合(領域), 出力集合(値域)$\rangle$ という形式で表現する。例えば、D_e の領域にあるインプットから D_t の値域にあるアウトプットに変換する関数であれば、これをタイプ $\langle e, t \rangle$ の言語表現と呼び、次のように記す。

(29) $[\![\textbf{run(s)}]\!] \in D_{\langle e,t \rangle}$

入力される意味に制限があることを明示したい場合、(30)b のように表す。出力も明記する場合、(30)c のようにする。

(30) a. $[\![\textbf{run(s)}]\!] = \lambda x \quad . \text{TRUE} \quad , \text{iff } x \in D_{run}.$
b. $[\![\textbf{run(s)}]\!] = \lambda x \underline{\in D_e}. \text{TRUE} \quad , \text{iff } x \in D_{run}.$
c. $[\![\textbf{run(s)}]\!] = \lambda x \underline{\in D_e}. \text{TRUE} \underline{\in D_t}, \text{iff } x \in D_{run}.$

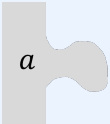
↑ 領域条件　↑ 値域条件

分かり切っているときは、領域条件である $\in D_e$ を省略して (30)a のように書くこともある。

付録：具体例 1

基本タイプ１ e

個体（individual）を指示する表現の意味タイプ

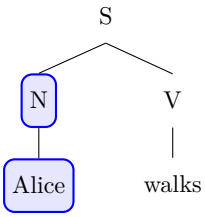


ここがポイント！ 個体（entity）というタイプ

(31) タイプ e: 個体（entity）というタイプ

a. $D_e :=$ 個体の集合（例：$D_e = \{alice, bob, chris, ...\}$ ）

b. $[\![$ **Alice** $]\!] \in D_e$ 　固有名詞



ここがポイント！ 意味計算

$[\![$ **S** $]\!] =$ $[\![$ **Alice walks.** $]\!]$

※統語構造をもとに分解（合成性の原則）

$=$ $[\![$ **walks** $]\!]([\![$ **Alice** $]\!])$

※ Alice の意味は、その指示対象

$=$ $[\![$ **walks** $]\!](a)$

※ run の意味は、個体を取って真偽を返す関数

$= [\lambda x.\text{TRUE, iff } x \in D_{walk}](a)$

※xにaを代入

$=$ TRUE, iff $a \in D_{walk}$

付録：具体例 2

基本タイプ 2 t

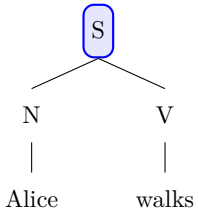
真理値（truth value）を表すタイプ
すなわち、真偽が判定できるもの

ここがポイント！　真理値（truth value）というタイプ

(32) タイプ t：真理値（truth value）というタイプ

a. $D_t := \{$ TRUE, FALSE $\}$（別の表し方：$D_t := \{ 1, 0 \}$）

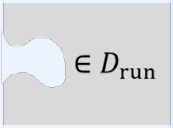
b. $[\![$ **Alice walks** $]\!] \in D_t$　　文（命題）



付録：具体例 3

派生タイプ1 $\langle e, t \rangle$

個体（type e）を入力として取り、真理値（type t）を返す関数のタイプ



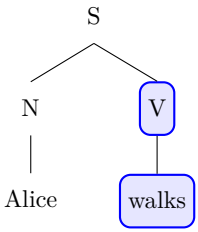
ここがポイント！　個体を取ったら命題が完成する

個体を取って真理値を返す関数」のタイプ

$D_{\langle e,t \rangle}$:= 個体を取って真理値を返す関数の集合

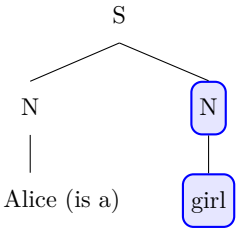
(33) a. $[\![$ **walk** $]\!] \in D_{\langle e,t \rangle}$

自動詞（一項述語）



b. $[\![$ **girl** $]\!] \in D_{<e,t>}$

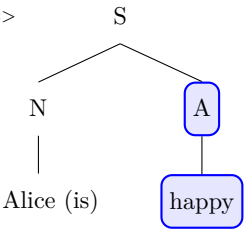
普通名詞（一項述語）



すべての普通名詞が一項名詞化と言うと議論がある。girl と違って daughter なら「誰が誰の娘か」という複数の登場人物（人間関係）を背景に意味が成り立つからだ。

c. $[\![$ **happy** $]\!] \in D_{<e,t>}$

形容詞（一項述語）



Chapter 4

ここがポイント！ 意味計算：自動詞 walk

$[\![\text{S}]\!] = [\![\textbf{Alice walks.}]\!]$

※統語構造をもとに分解（合成性の原則）

$= [\![\textbf{V}]\!]([\![\textbf{N}]\!])$

※ V、N の意味はそれにぶら下がる単語の意味

$= [\![\textbf{walk}]\!]([\![\textbf{Alice}]\!])$

※ Alice の意味は、その指示対象

$= [\![\textbf{walk}]\!](a)$

※ girl の意味は、個体を取って真偽を返す関数

$= [\lambda x.\text{TRUE, iff } x \in D_{walk}](a)$

※xにaを代入

$= \text{TRUE, iff } a \in D_{walk}$

ここがポイント！ 意味計算：普通名詞 girl

$[\![\text{S}]\!] = [\![\textbf{Alice is a girl.}]\!]$

※この際 is と a は意味に寄与しないものとして無視

$= [\![\textbf{girl}]\!]([\![\textbf{Alice}]\!])$

※ Alice の意味は、その指示対象

$= [\![\textbf{girl}]\!](a)$

※ girl の意味は、個体を取って真偽を返す関数

$= [\lambda x.\text{TRUE, iff } x \in D_{girl}](a)$

※xにaを代入

$= \text{TRUE, iff } a \in D_{girl}$

ここがポイント！ 意味計算：形容詞 happy

$[\![\text{S}]\!] = [\![\textbf{Alice is happy.}]\!]$

※この際 is は意味に寄与しないものとして無視

$= [\![\textbf{happy}]\!]([\![\textbf{Alice}]\!])$

$= [\![\textbf{happy}]\!](a)$

$= [\lambda x.\text{TRUE, iff } x \in D_{happy}](a)$

$= \text{TRUE, iff } a \in D_{happy}$

付録：具体例 4

派生タイプ2 $\langle e, \langle e, t \rangle \rangle$

個体（e）を入力として取り、「個体 → 真理値」の関数 $\langle e, t \rangle$ を返す関数のタイプ。

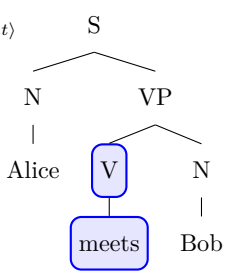
括弧が重なると分かりにくいので、$\langle e, t \rangle$ をまとめて et と表記することがある。つまり、$\langle e, \langle e, t \rangle \rangle$ のことを、$\langle e, et \rangle$ と書くこともある。

$D_{\langle e, et \rangle}$:= 個体を取りタイプ et の関数を返す関数の集合

ここがポイント！ 二つ固有名詞を取ったら命題が完成する。

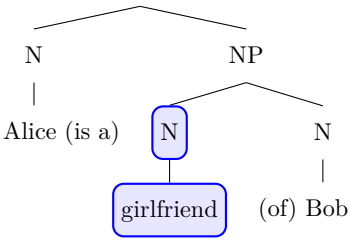
「個体を取ってタイプ $< e, t >$ の関数を返す関数」というタイプ。

(34) a. $[\![$ **meets** $]\!] \in D_{\langle e, et \rangle}$



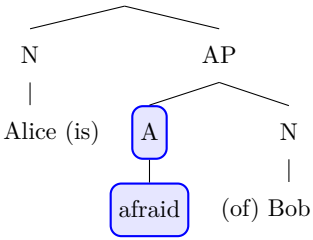
他動詞（二項述語）

b. $[\![$ **girlfriend** $]\!] \in D_{\langle e, et \rangle}$



普通名詞（二項述語）

c. $[\![$ **afraid** $]\!] \in D_{\langle e, et \rangle}$



形容詞（二項述語）

ここがポイント！ 意味計算：他動詞 meet

$$
\begin{aligned}
[\![\mathbf{S}]\!] &= [\![\textbf{Alice meets Bob.}]\!] \\
&\quad \text{※統語構造をもとに分解（合成性の原則）} \\
&= [\![\mathbf{VP}]\!]([\![\mathbf{N}]\!]) \\
&\quad \text{※ Alice の意味は、その指示対象} \\
&= [\![\mathbf{VP}]\!](a) \\
&\quad \text{※ VP の中を展開} \\
&= \Big[[\![\mathbf{V}]\!]([\![\mathbf{N}]\!])\Big](a) \\
&\quad \text{※ V、N の意味はそれにぶら下がる単語の意味} \\
&= \Big[[\![\textbf{meet}]\!]([\![\textbf{Bob}]\!])\Big](a) \\
&\quad \text{※ meet の意味は、個体を取って、さらに個体を取り} \\
&\quad\quad \text{真偽を返す関数} \\
&= \Big[[\lambda y.\lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{meet}(x,y)]\,([\![\textbf{Bob}]\!])\Big](a) \\
&\quad \text{※ Bob の意味は、その指示対象} \\
&= \Big[[\lambda y.\lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{meet}(x,y)]\,(b)\Big](a) \\
&\quad \text{※}y\text{に}b\text{を代入} \\
&= \Big[\quad \lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{meet}(x,b) \quad\Big](a) \\
&\quad \text{※}x\text{に}a\text{を代入} \\
&= \qquad \mathrm{TRUE},\ \text{iff } \mathtt{meet}(a,b).
\end{aligned}
$$

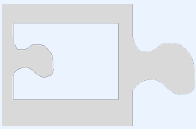
ここがポイント！ 意味計算：普通名詞 girlfriend

$$
\begin{aligned}
[\![\mathbf{S}]\!] &= [\![\textbf{Alice is a girlfriend of Bob.}]\!] \\
&\quad \text{※この際 is と a と of は意味に寄与しないものとして無視} \\
&= [\![\mathbf{NP}]\!]([\![\mathbf{N}]\!]) \\
&= [\![\mathbf{NP}]\!](a) \\
&= \Big[[\![\mathbf{N}]\!]([\![\mathbf{N}]\!])\Big](a) \\
&= \Big[[\![\textbf{girlfriend}]\!]([\![\textbf{Bob}]\!])\Big](a) \\
&= \Big[[\lambda y.\lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{girlfriend}(x,y)]\,([\![\textbf{Bob}]\!])\Big](a) \\
&= \Big[[\lambda y.\lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{girlfriend}(x,y)]\,(b)\Big](a) \\
&= \Big[\quad \lambda x.\mathrm{TRUE},\ \text{iff } \mathtt{girlfriend}(x,b) \quad\Big](a) \\
&= \qquad \mathrm{TRUE},\ \text{iff } \mathtt{girlfriend}(a,b).
\end{aligned}
$$

付録：具体例 5

派生タイプ３ $\langle\langle e,t\rangle,e\rangle$

性質 $\langle e,t\rangle$ を入力として取り、個体 e を返す（その性質を満たす個体を “選び出す”）関数

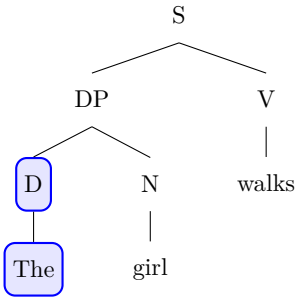


ここがポイント！　普通名詞から “固有名詞” を作る

「タイプ et の関数を取ってタイプ e の個体を返す関数」のタイプ

$D_{\langle\langle e,t\rangle,e\rangle}$:= タイプ et の関数を取ってタイプ e の個体を返す関数の集合

(35) $[\![$ **the** $]\!] \in D_{\langle\langle e,t\rangle,e\rangle}$　　定冠詞



付録：具体例 6

派生タイプ４ $\langle et, et \rangle$

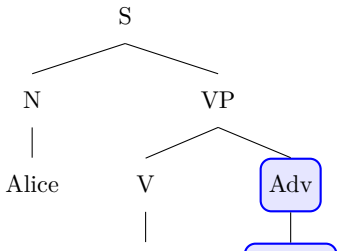
性質 $\langle e, t \rangle$ を入力として取り、アップデートした別の性質 $\langle e, t \rangle$ を作る関数

ここがポイント！ タイプを変えずに、情報を追加する関数

「タイプ et の関数を取ってタイプ et の関数を返す関数」というタイプ

(36) $[\![$ **quickly** $]\!] \in D_{\langle et,et \rangle}$

副詞



$D_{\langle et,et \rangle}$:= タイプ et の関数を取ってタイプ et の関数を返す関数の集合

ここがポイント！ 意味の計算

$[\![$ **S** $]\!] =$ $[\![$ **Alice walks quickly.** $]\!]$

※統語構造をもとに分解（合成性の原則）

$=$ $[\![$ **walks quickly** $]\!]([\![$ **Alice** $]\!])$

※統語構造をもとに分解（合成性の原則）

$=$ $\left[[\![\textbf{quickly}]\!]([\![\textbf{walks}]\!])\right](a)$

※ quickly が walk(s) を取る。
quickly の具体的な中身は後日紹介。

付録：具体例 7

派生タイプ 5 $\langle et, \langle et, t\rangle\rangle$

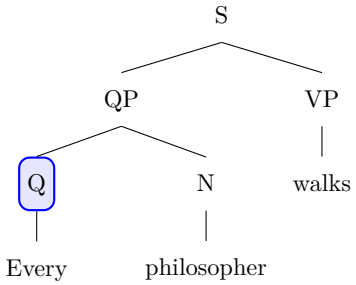
性質 $\langle e, t\rangle$ を 1 つ取り、さらにもう 1 つ性質 $\langle e, t\rangle$ を取って、真理値 t を返す関数

ここがポイント！

「タイプ et の関数を取ってタイプ $\langle et, t\rangle$ の関数を返す関数」というタイプ

$D_{\langle et,\langle et,t\rangle\rangle} :=$ タイプ et の関数を取ってタイプ $\langle et, t\rangle$ の関数を返す関数の集合

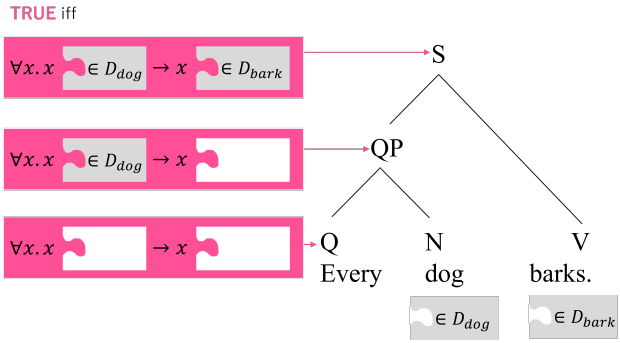
(37) $[\![$ **every** $]\!] \in D_{\langle et,\langle et,t\rangle\rangle}$ 量化詞



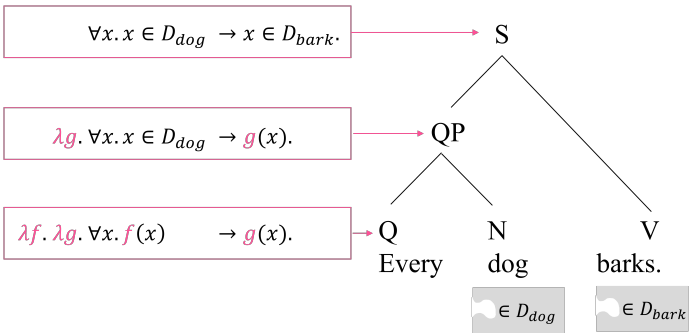
ここがポイント！ ラムダ演算子の使い方

1. “空欄” となるべき部分を記号（例：f や g）で表す。
2. 変項であるものは、λ をつけて λf のように表し、それを記号列の先頭に置く。
3. 複数 λ で束縛された変項がある場合、先頭から順に代入。

■いままで



■これから



どんな文でも、すべての（平叙）文の意味は最終的に「『真』または『偽』のいずれかだ」という点は、平叙文が and や if などと組み合わされて、命題論理を構築するという点を踏まえるとぜひ捉えたい側面です。しかし、it is raining も it is snowing もそれが「『真』または『偽』を表す」という点では共通していますが、別の次元では全然違うことを言っているという点も、また捉えたい側面です。では何が違っているのかというと、いつ「真」になるのかという条件です（真理条件）。そして、それは、最終的には用いられた単語である snow と rain という言葉の違いに帰せられます。

	ジグソーパズル	平叙文
完成したら	（内容に関係なく）長方形	（内容に関係なく）真または偽
		⟦ 任意の S ⟧ =T or F
差異性	描かれた模様	真理条件
		iff $a \in D_{run}$

ジグソーパズルは、たとえどんな絵柄であっても、完成すれば必ず一定の長方形に収まります。しかし、パズルの本質はそうした外枠の共通性だけでなく、個々のピースが織りなす模様、つまり「何が描かれているか」という固有のコンテンツにあります。文の分析もこれと同様です。文という形式が持つ共通の振る舞い（外延）と、その文だけが持つ固有の意味内容（真理条件）。私たちは、この二面性を同時に捉えるアプローチを実践しています。